

LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 4: MENGOPERASIKAN DATABASE DAN GENERATE DATA
OTOMATIS DENGAN LARAVEL



Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS
TAHUN AJARAN 2026

1. Tujuan

Adapun tujuan yang ingin dicapai pada praktikum tugas Modul 4 ini adalah sebagai berikut:

1. Menghubungkan project LaporBanjir dari Modul 3 dengan database MySQL melalui file .env, supaya laporan yang dikirim warga tidak lagi hilang ketika halaman ditutup.
2. Membuat migration tabel laporan dan menjalankannya sehingga tabel beserta kolomnya terbentuk di database db_laporbanjir.
3. Membuat model Laporan yang terhubung dengan tabel laporan melalui properti \$table dan \$fillable.
4. Mengubah method penyimpanan pada controller supaya data dari form pelaporan disimpan ke tabel memakai Laporan::create().
5. Mengubah halaman daftar laporan supaya mengambil data dari database dengan Laporan::all(), dengan tampilan yang tetap memakai layout, partial kartu, dan penentuan status genangan seperti pada Modul 3.
6. Membuat seeder LaporanSeeder berbasis Faker dengan locale id_ID untuk mengisi tabel dengan 20 data laporan contoh.
7. Menganalisis perbedaan aplikasi sebelum dan sesudah memakai database, termasuk kendala yang muncul selama pengerjaan.

2. Alat dan Bahan

Praktikum ini dikerjakan menggunakan satu buah laptop dengan prosesor Intel Core i7-13650HX dan sistem operasi Windows 11 Pro. Perangkat lunak yang digunakan meliputi Laragon sebagai lingkungan pengembangan web lokal beserta Apache dan MySQL di dalamnya, PHP versi 8.3.30, Composer versi 2.10.3, Laravel Framework versi 13.33.0, MySQL versi 8.4.3 sebagai server database, phpMyAdmin versi 5.2.3 untuk mengelola database, Visual Studio Code sebagai teks editor, Git sebagai version control, dan Google Chrome sebagai browser untuk menguji aplikasi.

3. Langkah-Langkah Praktikum

Tugas praktikum yang saya kerjakan adalah nomor 1 pada bagian Tugas Praktikum Modul 4, yaitu melanjutkan aplikasi LaporBanjir dari Modul 3 supaya setiap laporan yang dikirim warga benar-benar tersimpan di database. Pada Modul 3 data laporan masih ditulis sebagai array di dalam controller, sehingga isinya hilang begitu halaman ditutup. Bagian yang saya ubah adalah cara penyimpanan dan pengambilan datanya, lalu saya tambahkan seeder untuk mengisi tabel dengan data contoh. Alur aplikasinya tetap sama seperti sebelumnya, yaitu form pelaporan dikirim ke route, diproses controller, disimpan lewat model, dan ditampilkan kembali pada halaman daftar.

3.1 Konsep Migration pada Laravel

Migration adalah fitur Laravel yang mencatat setiap perubahan struktur database dalam bentuk file PHP. Setiap file migration berisi dua method, yaitu `up()` yang dijalankan ketika migration dieksekusi untuk membuat atau mengubah tabel, dan `down()` yang dijalankan ketika rollback untuk membatalkan perubahan tersebut. Karena perubahannya tersimpan sebagai file, struktur database bisa dibangun ulang di komputer lain cukup dengan menjalankan `php artisan migrate`, tanpa perlu membuat tabel satu per satu lewat phpMyAdmin. Cara ini juga memudahkan kerja berkelompok karena file migration dapat dibagikan ke anggota tim lalu dijalankan pada database masing-masing. Fitur rollback berguna ketika perubahan terakhir ingin dibatalkan.

3.2 Konfigurasi Koneksi Database

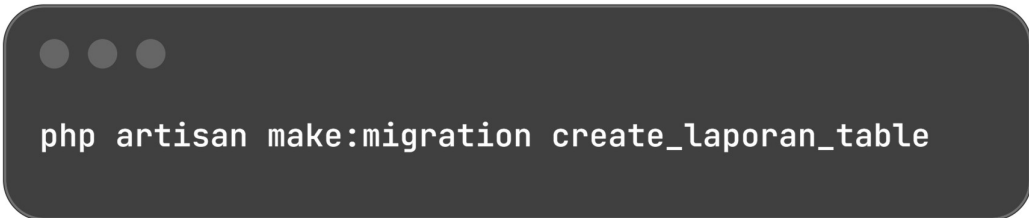
Apache dan MySQL dijalankan dari Laragon, lalu database baru bernama `db_laporbanjir` dibuat melalui phpMyAdmin. Setelah database tersedia, project dihubungkan ke database tersebut dengan mengubah bagian koneksi pada file `.env` di folder utama project.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laporbanjir
DB_USERNAME=root
DB_PASSWORD=
```

Nilai DB_CONNECTION diisi mysql karena server yang dipakai adalah MySQL, DB_HOST berisi alamat server (127.0.0.1), DB_PORT berisi port MySQL (3306), DB_DATABASE berisi nama database (db_laporbanjir), DB_USERNAME berisi root, dan DB_PASSWORD dibiarkan kosong karena instalasi Laragon yang saya pakai tidak memberi password pada user root. Pada Laravel versi terbaru nilai bawaan DB_CONNECTION adalah sqlite, jadi baris ini harus diubah menjadi mysql. Kalau perubahan pada .env belum terbaca, Laravel menyediakan perintah php artisan config:clear untuk menghapus cache konfigurasi.

3.3 Membuat Migration Tabel Laporan

Tabel laporan dibuat lewat file migration di folder database/migrations. File migration tersebut dibuat dengan perintah berikut:



```
php artisan make:migration create_laporan_table
```

Perintah itu menghasilkan file baru dengan nama yang diawali tanggal dan waktu pembuatan, dalam pengerjaan ini 2026_10_05_143121_create_laporan_table.php. Awalan waktu tersebut dipakai Laravel untuk menentukan urutan eksekusi migration. Isi filenya kemudian diganti menjadi seperti berikut:

```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('laporan', function (Blueprint $table) {
15             $table->id();
16             $table->string('nama_pelapor');
17             $table->string('lokasi');
18             $table->integer('tinggi_genangan');
19             $table->date('tanggal_kejadian');
20             $table->timestamps();
21         });
22     }
23
24     /**
25      * Reverse the migrations.
26      */
27     public function down(): void
28     {
29         Schema::dropIfExists('laporan');
30     }
31 };

```

Method up() membuat tabel laporan dengan tujuh kolom. Kolom id dibuat lewat \$table->id() sebagai primary key, nama_pelapor dan lokasi bertipe string, tinggi_genangan bertipe integer karena isinya angka dalam satuan sentimeter, tanggal_kejadian bertipe date, dan timestamps() menambahkan kolom created_at serta updated_at. Method down() menghapus tabel laporan ketika migration di-rollback. Setelah itu migration dijalankan dengan perintah berikut:



```
php artisan migrate
```

Selain tabel laporan, perintah `php artisan migrate` juga menjalankan migration bawaan Laravel yang membuat tabel `users`, `cache`, dan `jobs`, serta tabel `migrations` sebagai pencatat migration yang sudah dieksekusi. Hasil pengecekan struktur tabel di `phpMyAdmin` ditunjukkan pada Gambar 5.

3.4 Membuat Model Laporan

Model dibuat dengan perintah berikut:



```
php artisan make:model Laporan
```

Perintah itu menghasilkan file `app/Models/Laporan.php`. Isinya kemudian diganti menjadi seperti berikut:

```

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Laporan extends Model
8  {
9      protected $table = 'laporan';
10
11     protected $fillable = [
12         'nama_pelapor',
13         'lokasi',
14         'tinggi_genangan',
15         'tanggal_kejadian',
16     ];
17 }

```

Properti \$table diisi 'laporan' karena secara bawaan Laravel menganggap nama tabel adalah bentuk jamak bahasa Inggris dari nama model, sehingga tanpa properti ini model Laporan akan diarahkan ke tabel laporans. Properti \$fillable berisi daftar kolom yang boleh diisi sekaligus lewat satu perintah seperti Laporan::create([...]). Kolom yang tidak tercantum di \$fillable ditolak Laravel dengan galat MassAssignmentException.

3.5 Mengubah Controller agar Data Tersimpan ke Database

Pada Modul 3, method store() hanya mengoper data form ke halaman konfirmasi dan method index() mengisi array \$laporans dengan empat data contoh. Kedua method itu sekarang diubah supaya berhubungan dengan tabel laporan. Isi file app/Http/Controllers/LaporanController.php menjadi seperti berikut:

```

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Laporan;
6  use Illuminate\Http\Request;
7
8  class LaporanController extends Controller
9  {
10     // Tampilkan form pelaporan banjir.
11     public function create()
12     {
13         return view('laporan.create');
14     }
15
16     // Simpan data dari form ke tabel laporan, lalu tampilkan halaman konfirmasi.
17     public function store(Request $request)
18     {
19         $laporan = Laporan::create([
20             'nama_pelapor' => $request->input('nama_pelapor'),
21             'lokasi' => $request->input('lokasi'),
22             'tinggi_genangan' => $request->input('tinggi_genangan'),
23             'tanggal_kejadian' => $request->input('tanggal_kejadian'),
24         ]);
25
26         return view('laporan.konfirmasi', ['laporan' => $laporan]);
27     }
28
29     // Daftar laporan diambil dari tabel laporan.
30     public function index()
31     {
32         $laporans = Laporan::all();
33
34         return view('laporan.index', ['laporans' => $laporans]);
35     }
36 }

```

Method `store()` menyimpan data yang dikirim form memakai `Laporan::create()`, lalu mengirim baris hasil penyimpanan itu ke view konfirmasi. Nilai dari form dibaca lewat `$request->input()`, dan nama setiap input disamakan dengan nama kolom tabel supaya tidak ada data yang tertukar. Method `index()` mengganti array contoh dengan `Laporan::all()` sehingga seluruh isi tabel laporan dikirim ke view daftar. Method `create()` tetap hanya mengembalikan view form.

3.6 Mendefinisikan Route

Route pada file `routes/web.php` masih memakai alamat yang sama seperti Modul 3:

```
1 <?php
2
3 use App\Http\Controllers\LaporanController;
4 use Illuminate\Support\Facades\Route;
5
6 Route::get('/', [LaporanController::class, 'create'])->name('laporan.create');
7 Route::get('/laporan', [LaporanController::class, 'index'])->name('laporan.index');
8 Route::post('/laporan', [LaporanController::class, 'store'])->name('laporan.store');
```

Route GET / menampilkan form pelaporan, route POST /laporan memproses pengiriman form, dan route GET /laporan menampilkan daftar laporan. Ketiganya diberi nama lewat ->name() sehingga bisa dipanggil dari Blade dengan route('laporan.create'), route('laporan.store'), dan route('laporan.index'). Route POST /laporan dan route GET /laporan memakai alamat yang sama, tetapi keduanya tidak bentrok karena method HTTP-nya berbeda.

3.7 Menyesuaikan View Form, Konfirmasi, dan Daftar

Tiga file view dari Modul 3 disesuaikan karena datanya sekarang berupa objek Eloquent. File pertama adalah resources/views/laporan/create.blade.php yang dipakai untuk menginput laporan:

```

1  @extends('layouts.app')
2
3  @section('title', 'Form Pelaporan Banjir')
4
5  @section('content')
6      <h1>Form Pelaporan Banjir</h1>
7      <p class="muted">Laporkan kejadian banjir di wilayah Anda.</p>
8
9      <form method="POST" action="{{ route('laporan.store') }}">
10         @csrf
11
12         <div class="field">
13             <label for="nama_pelapor">Nama Pelapor</label>
14             <input type="text" id="nama_pelapor" name="nama_pelapor"
15                 placeholder="Contoh: Budi Santoso">
16         </div>
17
18         <div class="field">
19             <label for="lokasi">Lokasi Kejadian</label>
20             <input type="text" id="lokasi" name="lokasi"
21                 placeholder="Contoh: Jl. Sukajadi No. 12, Bandung">
22         </div>
23
24         <div class="field">
25             <label for="tinggi_genangan">Tinggi Genangan Air (cm)</label>
26             <input type="number" id="tinggi_genangan" name="tinggi_genangan" min="1"
27                 placeholder="Contoh: 45">
28         </div>
29
30         <div class="field">
31             <label for="tanggal_kejadian">Tanggal Kejadian</label>
32             <input type="date" id="tanggal_kejadian" name="tanggal_kejadian">
33         </div>
34
35         <button type="submit" class="btn">Kirim Laporan</button>
36     </form>
37 @endsection

```

Form memakai method POST dengan action route('laporan.store') dan directive @csrf untuk melindungi pengiriman data. Dibanding Modul 3, ada satu isian tambahan, yaitu tanggal kejadian, karena kolom tanggal_kejadian ikut diminta pada tugas ini. Atribut name pada setiap input ditulis sama dengan nama kolom tabel dan isi \$fillable, sehingga nilainya bisa dibaca controller melalui \$request.

File kedua adalah resources/views/laporan/konfirmasi.blade.php yang tampil setelah data tersimpan:

```

1  @extends('layouts.app')
2
3  @section('title', 'Konfirmasi Laporan')
4
5  @section('content')
6      <h1>Konfirmasi Laporan</h1>
7
8      <x-alert
9          type="success"
10         message="Laporan banjir berhasil disimpan. Terima kasih atas laporan Anda." />
11
12     <div class="detail">
13         <p>Berikut data laporan yang baru saja Anda kirim:</p>
14         <dl>
15             <dt>Nama Pelapor</dt>
16             <dd>{{ $laporan->nama_pelapor }}</dd>
17
18             <dt>Lokasi Kejadian</dt>
19             <dd>{{ $laporan->lokasi }}</dd>
20
21             <dt>Tinggi Genangan</dt>
22             <dd>{{ $laporan->tinggi_genangan }} cm</dd>
23
24             <dt>Tanggal Kejadian</dt>
25             <dd>{{ $laporan->tanggal_kejadian }}</dd>
26         </dl>
27
28         <div class="actions">
29             <a href="{{ route('laporan.index') }}" class="btn">Lihat Daftar Laporan</a>
30             <a href="{{ route('laporan.create') }}" class="btn">Buat Laporan Lagi</a>
31         </div>
32     </div>
33 @endsection

```

Pesan berhasil disimpan tetap memakai komponen `<x-alert>` dengan type success seperti pada Modul 3. Data yang ditampilkan berasal dari baris yang baru saja disimpan ke tabel, sehingga setiap kolom dipanggil dengan tanda panah seperti `$laporan->nama_pelapor`, bukan `$laporan['nama_pelapor']`. Kolom tanggal kejadian ikut ditampilkan pada halaman ini.

File ketiga dan keempat adalah partial kartu laporan serta halaman daftar:

```

1 <article class="card">
2   <div class="card-head">
3     <strong>{{ $laporan->lokasi }}</strong>
4
5     @if ($laporan->tinggi_genangan < 30)
6       <span class="badge waspada">Waspada</span>
7     @elseif ($laporan->tinggi_genangan <= 70)
8       <span class="badge siaga">Siaga</span>
9     @else
10      <span class="badge awas">Bahaya</span>
11    @endif
12  </div>
13
14  <p><strong>Pelapor:</strong> {{ $laporan->nama_pelapor }}</p>
15  <p><strong>Tinggi genangan:</strong> {{ $laporan->tinggi_genangan }} cm</p>
16  <p><strong>Tanggal kejadian:</strong> {{ $laporan->tanggal_kejadian }}</p>
17 </article>

```

```

1 @extends('layouts.app')
2
3 @section('title', 'Daftar Laporan Banjir')
4
5 @section('content')
6   <h1>Daftar Laporan Banjir</h1>
7   <p class="muted">
8     Status genangan: kurang dari 30 cm = Waspada, 30 - 70 cm = Siaga,
9     lebih dari 70 cm = Bahaya.
10  </p>
11
12  @forelse ($laporans as $laporan)
13    @include('partials.laporan-card', ['laporan' => $laporan])
14  @empty
15    <p class="empty">Belum ada laporan banjir.</p>
16  @endforelse
17 @endsection

```

Partial resources/views/partials/laporan-card.blade.php masih menentukan status genangan dengan @if, @elseif, dan @else seperti pada Modul 3, hanya saja nilai tinggi_genangan sekarang dibaca dari objek hasil query. Keterangan waktu laporan pada Modul 3 saya ganti menjadi tanggal kejadian karena tabel laporan menyimpan tanggal kejadian, bukan waktu laporan. Halaman daftar pada resources/views/laporan/index.blade.php memakai @forelse untuk mengulang kartu dan @include untuk memanggil partial, lalu menampilkan pesan lewat blok @empty kalau tabel laporan masih kosong.

3.8 Menjalankan dan Menguji Aplikasi

Server pengembangan dijalankan dengan php artisan serve, lalu alamat `http://127.0.0.1:8000` dibuka di browser. Saya mengisi form dengan nama Andhika Yuridho, lokasi Dayeuhkolot, tinggi genangan 45 cm, dan tanggal kejadian 5 Oktober 2026, kemudian menekan tombol Kirim Laporan. Halaman konfirmasi muncul dengan pesan bahwa laporan berhasil disimpan, dan setelah tombol Lihat Daftar Laporan ditekan, data yang baru dikirim ikut tampil bersama data hasil seeder. Hasil pengujian ditunjukkan pada Gambar 1 sampai Gambar 4.

3.9 Membuat Seeder

Seeder dipakai untuk mengisi tabel dengan data contoh secara otomatis supaya halaman daftar tidak kosong saat diuji. File seeder dibuat dengan perintah berikut:



```
php artisan make:seeder LaporanSeeder
```

Perintah itu menghasilkan file `database/seeder/LaporanSeeder.php`. Di folder yang sama juga terdapat `DatabaseSeeder.php`, yaitu seeder utama bawaan Laravel yang bisa dipakai untuk memanggil beberapa seeder sekaligus.

3.10 Mengisi Data Dummy dengan Faker

Faker adalah library PHP yang sudah terpasang bersama Laravel dan bisa menghasilkan data palsu seperti nama orang, alamat, nomor telepon, dan tanggal. Isi file `LaporanSeeder.php` diganti menjadi seperti berikut:

```

1  <?php
2
3  namespace Database\Seeders;
4
5  use Faker\Factory as Faker;
6  use Illuminate\Database\Seeder;
7  use Illuminate\Support\Facades\DB;
8
9  class LaporanSeeder extends Seeder
10 {
11     /**
12      * Run the database seeds.
13      */
14     public function run(): void
15     {
16         $faker = Faker::create('id_ID');
17
18         $kecamatan = [
19             'Dayeuhkolot',
20             'Baleendah',
21             'Bojongsoang',
22             'Rancaekek',
23             'Majalaya',
24         ];
25
26         for ($i = 0; $i < 20; $i++) {
27             $tanggal = $faker->dateTimeBetween('-1 month', 'now');
28
29             DB::table('laporan')->insert([
30                 'nama_pelapor' => $faker->name,
31                 'lokasi' => $faker->randomElement($kecamatan),
32                 'tinggi_genangan' => $faker->numberBetween(10, 150),
33                 'tanggal_kejadian' => $tanggal->format('Y-m-d'),
34                 'created_at' => now(),
35                 'updated_at' => now(),
36             ]);
37         }
38     }
39 }

```

Faker::create('id_ID') membuat data dengan format Indonesia sehingga nama pelapor yang muncul terdengar lokal. Perulangan for menjalankan pengisian sebanyak 20 kali, randomElement([...]) memilih lokasi dari lima kecamatan di Kabupaten Bandung, numberBetween(10, 150) menghasilkan tinggi genangan acak dalam rentang 10 sampai 150 cm, dan dateTimeBetween('-1 month', 'now') menghasilkan tanggal kejadian acak dalam satu

bulan terakhir. Data dimasukkan lewat `DB::table('laporan')->insert([...])` yang memakai Query Builder, sedangkan kolom `created_at` dan `updated_at` diisi dengan `now()`.

Agar seeder ini bisa dipanggil tanpa opsi `--class`, `LaporanSeeder` didaftarkan pada method `run()` di `DatabaseSeeder.php`:

```
1  <?php
2
3  namespace Database\Seeders;
4
5  use Illuminate\Database\Seeder;
6
7  class DatabaseSeeder extends Seeder
8  {
9      /**
10       * Seed the application's database.
11       */
12     public function run(): void
13     {
14         $this->call([
15             LaporanSeeder::class,
16         ]);
17     }
18 }
```

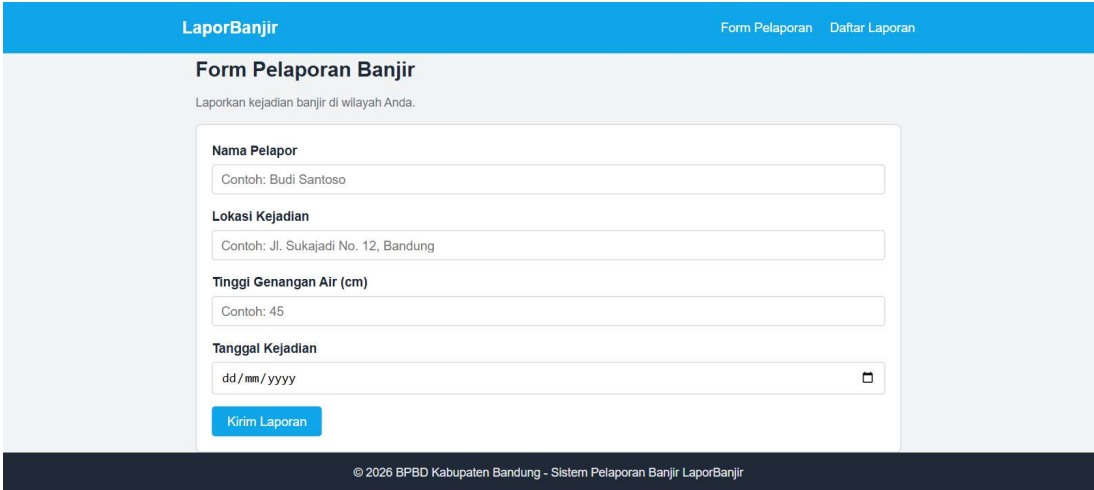
Tabel dibuat ulang sekaligus diisi dengan perintah berikut:

```
php artisan migrate:fresh --seed
```

Perintah `migrate:fresh` menghapus seluruh tabel lalu menjalankan ulang semua migration dari awal sehingga tabel laporan dibuat kembali dengan struktur terbaru, sedangkan opsi `--seed` menjalankan DatabaseSeeder setelah migration selesai. Seluruh data lama ikut terhapus, jadi perintah ini hanya cocok dipakai pada tahap pengembangan. Hasilnya, tabel laporan terisi 20 data dummy yang bisa dilihat di phpMyAdmin pada Gambar 6.

4. Hasil dan Dokumentasi Program

Berikut adalah tangkapan layar aplikasi LaporBanjir yang berjalan melalui php artisan serve pada alamat `http://127.0.0.1:8000` serta tampilan tabel laporan di phpMyAdmin.



Gambar 1. Halaman form pelaporan pada alamat /

Gambar 1 menampilkan halaman form yang dihasilkan oleh method `create()`. Formulirnya memuat empat isian, yaitu nama pelapor, lokasi kejadian, tinggi genangan air dalam satuan sentimeter, dan tanggal kejadian, dengan tombol Kirim Laporan di bagian bawah. Menu Form Pelaporan dan Daftar Laporan di header memakai named route, sedangkan footer menampilkan nama instansi penyelenggara.

LaporBanjir

Form Pelaporan

Daftar Laporan

Form Pelaporan Banjir

Laporkan kejadian banjir di wilayah Anda.

Nama Pelapor
Andhika Yuridho

Lokasi Kejadian
Dayeuhkolot

Tinggi Genangan Air (cm)
45

Tanggal Kejadian
05/10/2026

Kirim Laporan

© 2026 BPBD Kabupaten Bandung - Sistem Pelaporan Banjir LaporBanjir

Gambar 2. Form pelaporan yang sudah diisi data laporan

Gambar 2 memperlihatkan form yang sudah diisi dengan nama Andhika Yuridho, lokasi Dayeuhkolot, tinggi genangan 45 cm, dan tanggal kejadian 5 Oktober 2026. Kolom tanggal memakai input bertipe date sehingga tampilannya mengikuti format tanggal dari browser, sementara nilai yang dikirim ke controller tetap dalam format tahun-bulan-hari.

LaporBanjir

Form Pelaporan

Daftar Laporan

Konfirmasi Laporan

Success: Laporan banjir berhasil disimpan. Terima kasih atas laporan Anda.

Berikut data laporan yang baru saja Anda kirim:

Nama Pelapor	Andhika Yuridho
Lokasi Kejadian	Dayeuhkolot
Tinggi Genangan	45 cm
Tanggal Kejadian	2026-10-05

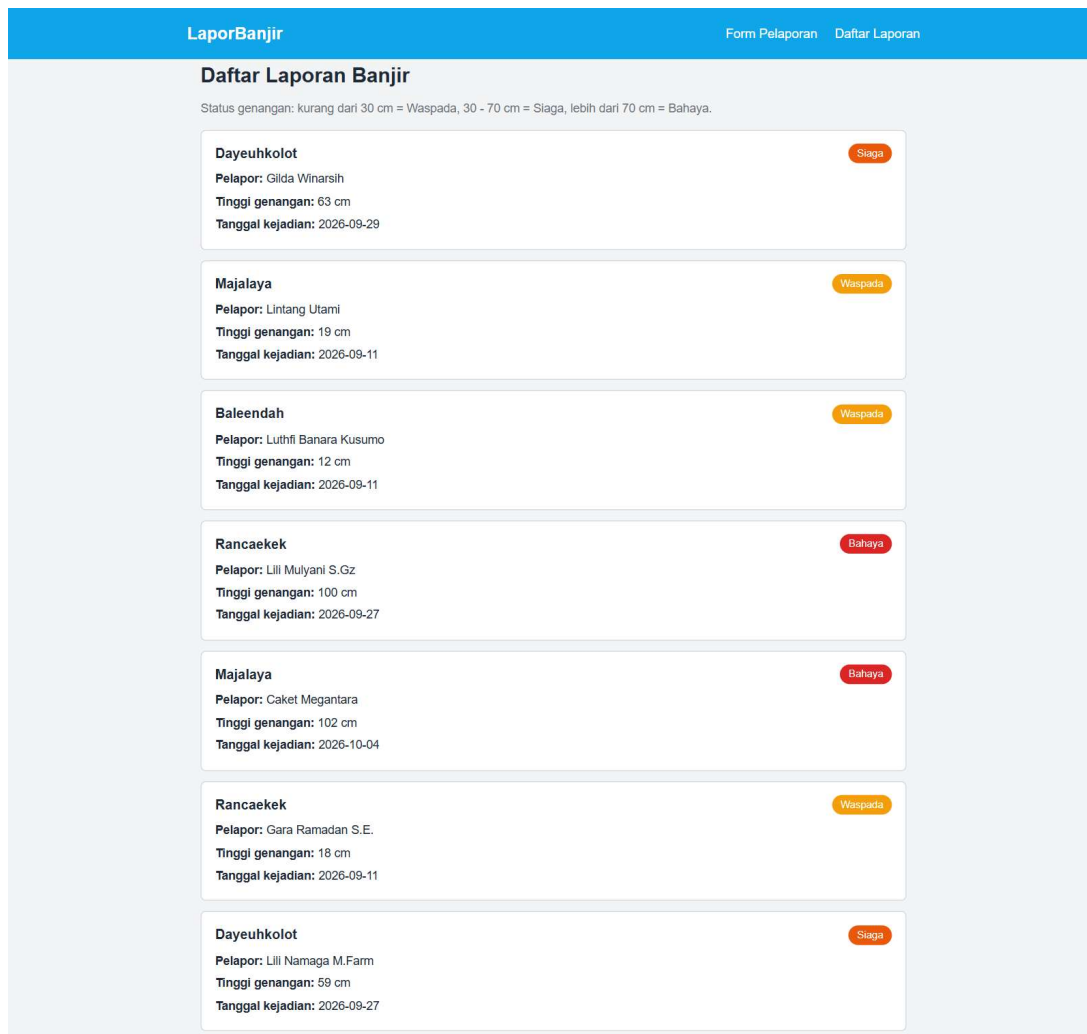
Lihat Daftar Laporan

Buat Laporan Lagi

© 2026 BPBD Kabupaten Bandung - Sistem Pelaporan Banjir LaporBanjir

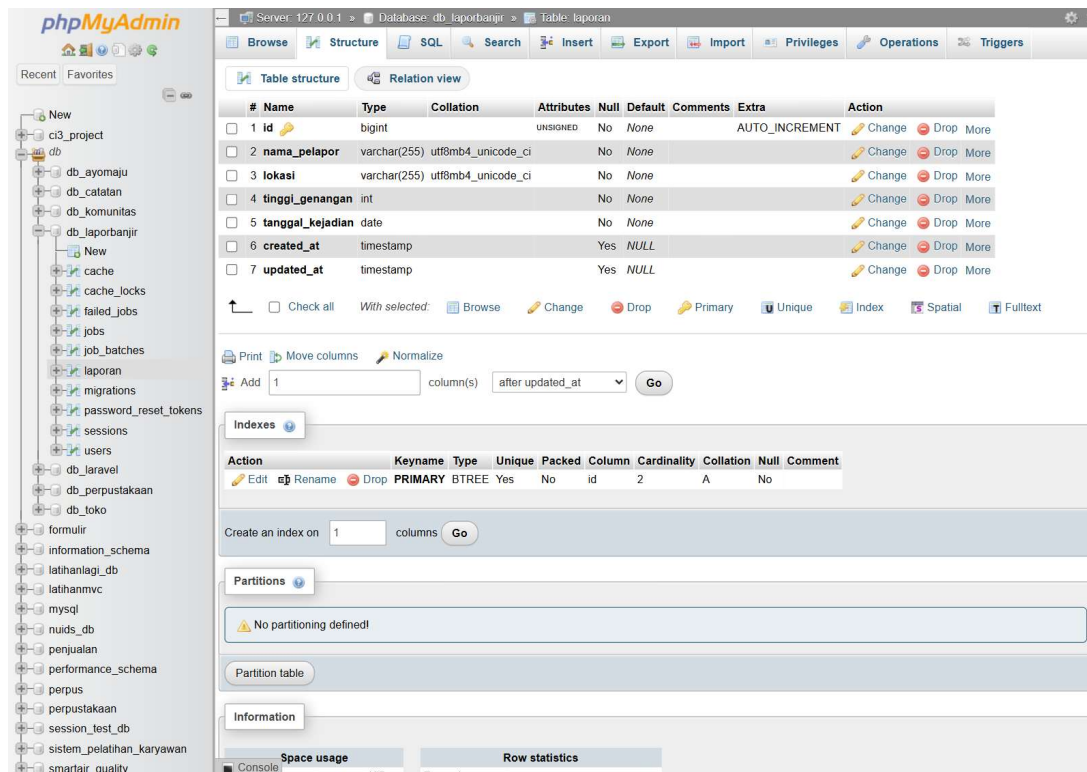
Gambar 3. Halaman konfirmasi setelah laporan disimpan

Gambar 3 diambil setelah tombol Kirim Laporan ditekan. Pesan Laporan banjir berhasil disimpan tampil di dalam komponen `<x-alert>` dengan border hijau karena atribut type berisi success, lalu di bawahnya tampil empat data laporan yang baru saja masuk ke tabel. Halaman ini dibuat oleh method `store()` dengan mengirim baris hasil `Laporan::create()` ke view konfirmasi.



Gambar 4. Halaman daftar laporan yang datanya berasal dari tabel laporan

Gambar 4 menampilkan kartu laporan pada alamat /laporan. Tujuh kartu teratas pada tangkapan layar ini memuat tiga status yang berbeda, yaitu Waspada untuk genangan di bawah 30 cm, Siaga untuk 30 sampai 70 cm, dan Bahaya untuk di atas 70 cm. Setiap kartu dirender oleh partial laporan-card, dan jumlah kartu mengikuti isi tabel laporan karena datanya diambil dengan `Laporan::all()`, bukan dari array contoh seperti pada Modul 3.



Gambar 5. Struktur tabel laporan pada phpMyAdmin

Gambar 5 memperlihatkan struktur tabel laporan yang terbentuk dari migration. Tabel ini memiliki tujuh kolom, yaitu id bertipe bigint dengan AUTO_INCREMENT, nama_pelapor dan lokasi bertipe varchar(255), tinggi_genangan bertipe int, tanggal_kejadian bertipe date, serta created_at dan updated_at bertipe timestamp. Pada bagian Indexes terlihat PRIMARY bertipe BTREE pada kolom id.

Showing rows 0 - 20 (21 total, Query took 0.0003 seconds)

SELECT * FROM `laporan`

Number of rows: 25 Filter rows: Search this table Sort by key: None

	id	nama_pelapor	lokasi	tinggi_genangan	tanggal_kejadian	created_at	updated_at
☐	1	Gilda Winarsih	Dayeuhkolot	63	2026-09-29	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	2	Lintang Utami	Majalaya	19	2026-09-11	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	3	Luthfi Banara Kusumo	Baleendah	12	2026-09-11	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	4	Lili Mulyani S.Gz	Rancaekek	100	2026-09-27	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	5	Caket Meganlara	Majalaya	102	2026-10-04	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	6	Gara Ramadan S.E.	Rancaekek	18	2026-09-11	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	7	Lili Namaga M Farm	Dayeuhkolot	59	2026-09-27	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	8	Raisa Padmasari	Baleendah	32	2026-10-05	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	9	Radika Agus Latupono	Majalaya	84	2026-10-02	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	10	Ulya Handayani S.Farm	Bojongsong	83	2026-09-21	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	11	Violet Mardhiyah	Bojongsong	118	2026-09-10	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	12	Nasab Hidayanto S.Psi	Baleendah	78	2026-09-06	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	13	Johan Sitompul M.Farm	Bojongsong	101	2026-10-01	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	14	Nrma Pranawa Habibi	Majalaya	44	2026-09-21	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	15	Muhammad Haryanto	Majalaya	45	2026-09-25	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	16	Dariati Narpati S.E.	Rancaekek	28	2026-09-14	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	17	Janet Rahmi Halimah	Rancaekek	56	2026-09-11	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	18	Mahmud Luis Sitorus	Bojongsong	16	2026-09-12	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	19	Pangestu Dabukke	Majalaya	86	2026-10-03	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	20	Makuta Kenes Hidayat S.E.I	Rancaekek	74	2026-10-03	2026-10-05 14:48:11	2026-10-05 14:48:11
☐	21	Andhika Yuridho	Dayeuhkolot	45	2026-10-05	2026-10-05 14:48:20	2026-10-05 14:48:20

Gambar 6. Data hasil seeder pada tabel laporan di phpMyAdmin

Gambar 6 menampilkan isi tabel laporan setelah seeder dijalankan dan satu data dikirim lewat form. phpMyAdmin melaporkan 21 baris dengan waktu query 0.0003 detik, yaitu 20 baris hasil LaporanSeeder dan satu baris terakhir dengan id 21 berisi data Andhika Yuridho yang saya input sendiri melalui form. Nama pelapor dan lokasi pada baris hasil seeder berbeda-beda karena datanya dibuat acak oleh Faker.

5. Analisis dan Pembahasan

Perbedaan paling terasa antara LaporBanjir versi Modul 3 dan versi Modul 4 ada pada masa hidup datanya. Pada Modul 3, method store() hanya mengoper isi form ke halaman konfirmasi dan method index() membaca array yang ditulis di dalam controller, sehingga laporan yang dikirim warga hilang begitu halaman ditutup atau server dijalankan ulang. Setelah memakai database, setiap laporan disimpan sebagai satu baris di tabel laporan dan tetap ada sampai dihapus, sehingga halaman daftar bisa menampilkan seluruh riwayat laporan yang pernah dikirim. Perubahan ini menjawab kebutuhan BPBD pada soal, yaitu agar laporan warga bisa direkap kapan saja.

Struktur tabel laporan dipilih menyesuaikan jenis data yang disimpan. Kolom id memakai `$table->id()` agar setiap baris punya primary key berupa angka yang bertambah otomatis. Kolom `nama_pelapor` dan `lokasi` memakai tipe string karena isinya teks pendek. Kolom `tinggi_genangan` memakai tipe integer karena isinya angka dalam satuan sentimeter dan nilainya dipakai untuk perbandingan pada penentuan status genangan, jadi menyimpan sebagai angka lebih tepat daripada teks. Kolom `tanggal_kejadian` memakai tipe date karena yang dicatat hanya tanggal kejadian, bukan jam. Kolom `timestamps()` menambahkan `created_at` dan `updated_at` yang diisi otomatis oleh Laravel, sehingga waktu penyimpanan data tidak perlu dicatat manual.

Migration dan seeder punya peran yang berbeda meskipun keduanya dijalankan lewat artisan. Migration mengurus struktur tabel, misalnya membuat tabel atau menambah kolom, dan hasilnya bertahan sampai ada migration baru yang mengubahnya. Seeder mengurus isi tabel, dan datanya bisa dibuat ulang kapan saja dengan menjalankan ulang perintahnya. Karena perannya berbeda, keduanya dipakai berurutan: migration dulu untuk membentuk tabel, baru seeder untuk mengisi data uji. Pada pengerjaan ini keduanya digabung dalam satu perintah, yaitu `php artisan migrate:fresh --seed`.

Satu hal yang harus disesuaikan ketika data berpindah dari array ke database adalah cara memanggil kolom di dalam view. Pada Modul 3, tiap laporan berupa array sehingga kolomnya dibaca dengan tanda kurung siku, misalnya `$laporan['lokasi']`. Hasil query Eloquent berupa objek, sehingga penulisannya berubah menjadi `$laporan->lokasi`. Bagian yang memakai array masih tetap jalan karena objek Eloquent mendukung akses seperti array, tetapi kolom yang tidak ada di tabel, seperti keterangan waktu laporan pada Modul 3, akan memunculkan galat karena kolomnya tidak ikut dibuat di migration. Karena itu keterangan tersebut saya ganti menjadi tanggal kejadian yang memang tersimpan di tabel.

Kendala yang muncul selama pengerjaan tercatat di file `storage/logs/laravel.log`. Pertama, muncul galat `SQLSTATE[42S02]: Base table or view not found: 1146 Table 'db_laporbanjir.sessions' doesn't exist` ketika aplikasi dibuka. Penyebabnya adalah `SESSION_DRIVER` pada `.env` berisi database, sedangkan migration belum dijalankan sehingga tabel `sessions` belum ada. Galat itu hilang setelah `php artisan migrate` dijalankan karena tabel `sessions` dibuat oleh migration bawaan Laravel. Kedua, halaman konfirmasi sempat gagal dengan galat `Undefined variable $laporan`. Variabel `$laporan` dipakai di dalam view konfirmasi, dan galat itu hilang setelah method `store()` mengirim baris hasil

Laporan::create() ke view tersebut. Setelah kedua perbaikan itu, seluruh alur form sampai database berjalan tanpa galat.

Ada dua penyesuaian yang saya lakukan terhadap ketentuan modul. Pertama, nama status untuk genangan di atas 70 cm saya tulis menjadi Bahaya, sedangkan modul menyebutnya Awas. Penamaan ini sudah dipakai sejak Modul 3 dan rentang pengukurannya tetap sama, yaitu di bawah 30 cm Waspada, 30 sampai 70 cm Siaga, dan di atas 70 cm Bahaya. Kedua, jumlah data dummy pada seeder saya tetapkan 20 baris, sedangkan modul meminta minimal 15 baris. Jumlah itu saya pilih supaya halaman daftar terisi cukup banyak kartu saat diuji.

6. Kesimpulan

Berdasarkan praktikum tugas Modul 4 ini, aplikasi LaporBanjir berhasil dihubungkan dengan database MySQL melalui file .env dengan DB_CONNECTION mysql dan database db_laporbanjir. Tabel laporan terbentuk dari migration dengan tujuh kolom, yaitu id, nama_pelapor, lokasi, tinggi_genangan, tanggal_kejadian, created_at, dan updated_at, dan model Laporan terhubung ke tabel tersebut melalui properti \$table dan \$fillable.

Alur form sampai database juga berjalan sesuai rancangan. Data yang dikirim dari create.blade.php diterima method store() pada LaporanController melalui route POST /laporan, disimpan ke tabel laporan dengan Laporan::create(), lalu ditampilkan kembali oleh method index() pada halaman daftar yang datanya diambil dengan Laporan::all(). Halaman konfirmasi tetap memakai komponen <x-alert>, dan penentuan status genangan pada partial kartu tetap memakai @if, @elseif, dan @else seperti pada Modul 3.

Seeder dengan Faker berhasil mengisi tabel dengan 20 data laporan berbahasa Indonesia, lengkap dengan variasi tinggi genangan yang membuat status Waspada, Siaga, dan Bahaya muncul bersamaan di halaman daftar. Dari praktikum ini saya jadi paham pembagian tugas antara migration dan seeder, pentingnya mencocokkan nama kolom dengan \$fillable dan atribut name pada form, serta perbedaan cara memanggil data array dan data hasil query Eloquent di dalam view.

7. Link Repository Github

<https://github.com/dhodhoo/pabw-andhika-yuridho/tree/main/pabw-4/praktikum4-ta>